

Experiment No.	6
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
Student Name	S.S.Janane
Register Number	3122247001024
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	30-8-2026

1 Objective

This experiment studies how Principal Component Analysis (PCA) affects the performance of ten different classifiers. Each model is trained and validated twice, once on the original features and once on the PCA-reduced features, using hyperparameter tuning and 5-fold cross-validation in both cases, so that the two settings can be compared fairly.

2 Problem Statement

Given a set of numerical features that describe breast tissue samples, the task is to classify each sample as malignant or benign. The input is a set of 30 numerical features per sample and the output is a binary label. The goal is to check whether reducing the number of features through PCA changes accuracy, F1-score, or stability across different types of models.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository, accessed through scikit-learn's load_breast_cancer
Number of Samples	569
Number of Features	30 numerical features
Number of Classes	2 (Malignant, Benign)
Class Distribution	357 Benign, 212 Malignant
Missing Values	None
Train-Test Split	80% train (455 samples), 20% test (114 samples), stratified

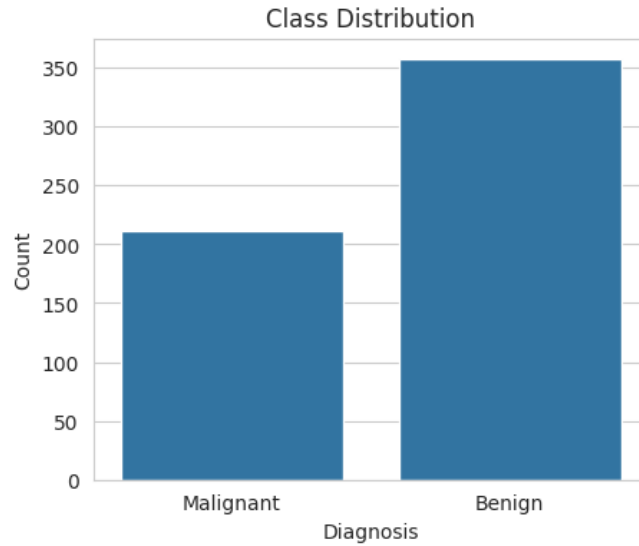


Figure 1: Class distribution of Malignant vs Benign samples

Inference

The data set is slightly imbalanced, with more benign than malignant samples. Stratified splitting was used for the train-test split and for every cross-validation fold, so this ratio is preserved throughout the experiment and does not bias the comparison between models.

4 Data Preprocessing

- Missing value handling: not required the dataset has no missing values
- Encoding: not required all features are numerical and the target is binary
- Feature scaling: StandardScaler was fit to the training data only and applied to both the train and the test sets
- Train-test split: 80-20 split with stratification on the target class
- PCA: applied on top of the scaled features, described in the next section

5 Principal Component Analysis

Setting	Chosen Components / Variance Target	Explained Variance (%)	Justification
With-PCA	10 components (95% variance target)	95.27	Keeps most of the information while roughly cutting the feature count in half

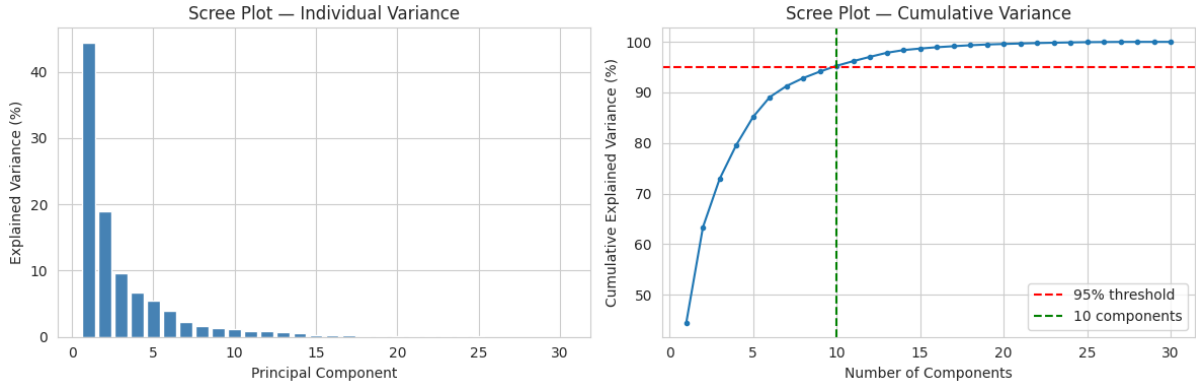


Figure 2: Scree plot: individual and cumulative explained variance by component

Inference

The individual variance plot shows that the first few components carry most of the information, and the curve flattens out quickly after that. The cumulative plot shows that 10 of the original 30 components are enough to cross the 95% variance mark, so the remaining 20 components add very little additional information.

6 Machine Learning Algorithms

Ten classifiers were evaluated: Support Vector Machine, Naive Bayes, k-Nearest Neighbors, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost and a Stacked Ensemble of SVM, Random Forest, and KNN with a Logistic Regression meta-learner. SVM and Logistic Regression are linear or margin-based models that work directly with the geometry of the feature space, so they are expected to respond differently to PCA compared to tree-based models such as Decision Tree, Random Forest, and the boosting methods, which build their decision boundaries by splitting on individual features rather than relying on distances or linear combinations.

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.x
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Standard CPU, no GPU used

8 Hyperparameter Selection

Model	Best Params (No-PCA)	Best Params (With-PCA)
SVM	C=0.1, gamma=scale, kernel=linear	C=10, gamma=0.01, kernel=rbf
Naive Bayes	var_smoothing=1e-9	var_smoothing=1e-9
KNN	k=3, manhattan, distance	k=5, euclidean, distance
Logistic Regression	C=0.1	C=0.1
Decision Tree	max_depth=3, min_samples_split=5	max_depth=5, min_samples_split=2
Random Forest	max_depth=10, n_estimators=100	max_depth=None, n_estimators=20
AdaBoost	learning_rate=1.0, n_estimators=100	learning_rate=1.0, n_estimators=50
Gradient Boosting	learning_rate=0.1, max_depth=2, n=100	learning_rate=0.1, max_depth=3, n=100
XGBoost	learning_rate=0.05, max_depth=3, n=100	learning_rate=0.05, max_depth=5, n=100
Stacking	final_estimator C=1	final_estimator C=1

All hyperparameters were selected using 5-fold stratified cross-validation on the training set, separately for the No-PCA and With-PCA feature sets.

9 Results

5-Fold Cross-Validation Results

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg No-PCA	Avg With-PCA
SVM	95.60	97.80	97.80	97.80	98.90	97.58	98.02
Naive Bayes	91.21	96.70	89.01	95.60	94.51	93.41	91.65
KNN	97.80	98.90	95.60	95.60	97.80	97.14	96.48
Logistic Regression	97.80	97.80	98.90	97.80	98.90	98.24	98.24
Decision Tree	92.31	93.41	92.31	92.31	93.41	92.75	92.31
Random Forest	96.70	95.60	93.41	96.70	98.90	96.26	96.04
AdaBoost	98.90	100.00	94.51	97.80	98.90	98.02	95.16
Gradient Boosting	95.60	98.90	94.51	96.70	97.80	96.70	96.26
XGBoost	98.90	96.70	94.51	96.70	98.90	97.14	96.48
Stacking	97.80	96.70	95.60	96.70	98.90	97.14	97.14

The fold values shown above are for the No-PCA setting, and the last two columns give the average accuracy for No-PCA and With-PCA respectively, so the change with PCA can be read directly from those two columns.

Test Set Results (Selected Models)

Model	Setting	Accuracy (%)	Precision	Recall	F1	AUC
SVM	No-PCA	98.25	0.986	0.986	0.986	0.994
SVM	With-PCA	95.61	0.986	0.944	0.965	0.996
Random Forest	No-PCA	95.61	0.959	0.972	0.966	0.994
Random Forest	With-PCA	93.86	0.958	0.944	0.951	0.986
XGBoost	No-PCA	94.74	0.946	0.972	0.959	0.993
XGBoost	With-PCA	93.86	0.958	0.944	0.951	0.993
Stacking	No-PCA	97.37	0.986	0.972	0.979	0.994
Stacking	With-PCA	95.61	0.972	0.958	0.965	0.991

10 Result Visualization

Cross-Validation Accuracy Comparison

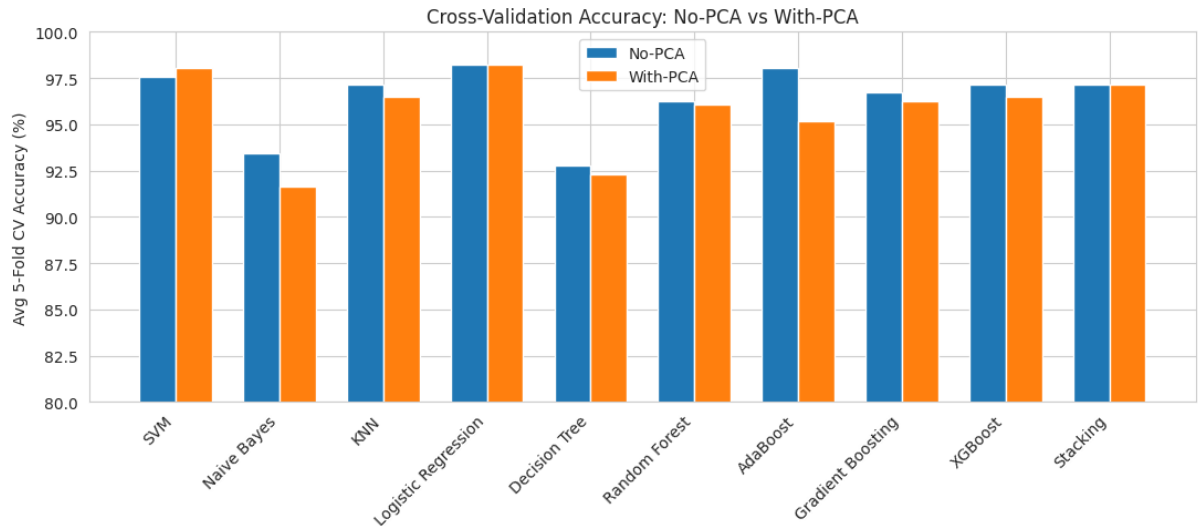


Figure 3: Average 5-fold CV accuracy for each model, No-PCA vs With-PCA

Inference

Most models sit close together, whether PCA is applied or not. SVM is the only model where the With-PCA bar is clearly taller than the No-PCA bar. AdaBoost shows the biggest drop, with its accuracy falling by close to three percentage points once PCA is applied.

Fold-to-Fold Variability

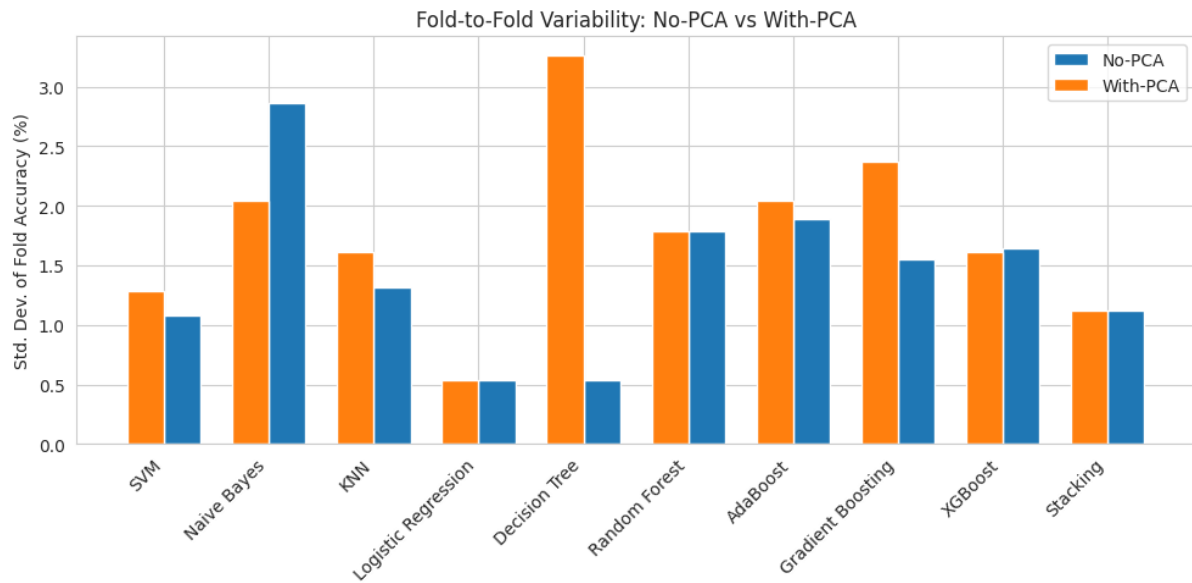


Figure 4: Standard deviation of fold accuracy for each model, No-PCA vs With-PCA

Inference

For several models, including Decision Tree, Gradient Boosting, and AdaBoost, the spread across folds becomes larger with PCA rather than smaller. Only Logistic Regression and Stacking keep exactly the same spread in both settings, since their best hyperparameters and scores did not change.

Confusion Matrices

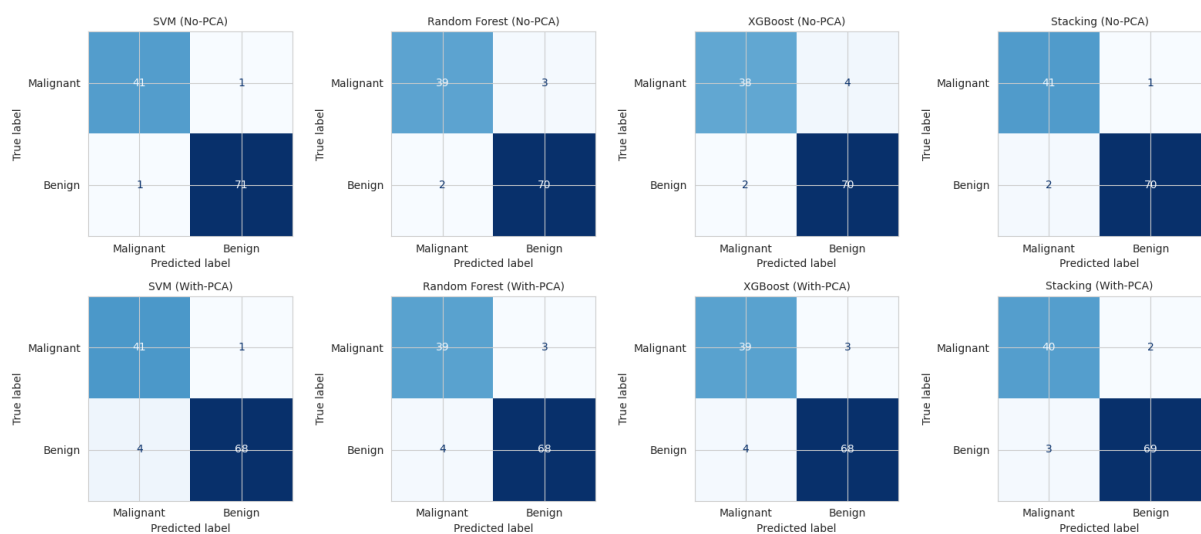


Figure 5: Confusion matrices for SVM, Random Forest, XGBoost and Stacking, No-PCA vs With-PCA

Inference

For all four models shown, the No-PCA version makes fewer errors or an equal number compared to the With-PCA version on the test set. SVM without PCA gets the closest to a perfect confusion matrix, while the With-PCA versions show a few more malignant cases that are misclassified as benign.

ROC Curves

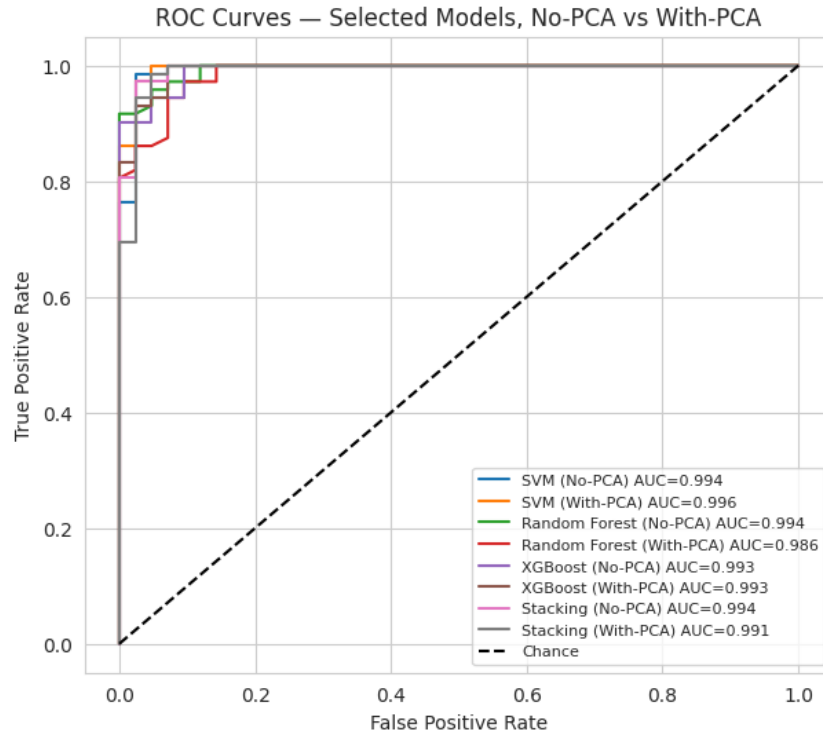


Figure 6: ROC curves for SVM, Random Forest, XGBoost and Stacking, No-PCA vs With-PCA

Inference

All eight curves stay close to the top left corner, which means that every model ranks the two classes well regardless of PCA. The AUC values are all above 0.98, and the small differences between the No-PCA and With-PCA curves for the same model are much smaller than the differences between different model types.

11 Discussion

Of the ten models tested, PCA only gave a clear benefit to SVM, whose cross-validation accuracy increased from 97.58% to 98.02%. Logistic Regression and Stacking were not affected, staying at exactly the same accuracy in both settings. The rest of the models, especially AdaBoost and Naive Bayes, performed worse with PCA. PCA also did not make the results more stable, the fold-to-fold spread went up for more models than it went down for. This line up with the fact that WDBC has only 30 features to begin with, which

is not extremely high-dimensional, so there was limited redundant information for PCA to usefully remove for most model types. Margin-based models such as SVM benefitted because PCA components are uncorrelated, which helps the margin computation, while boosting and probability-based models lost some of the specific feature signal on which they were relying.

12 Conclusion

Ten classifiers were tuned and evaluated with 5-fold cross-validation in the WDBC data set, both without PCA and with PCA reduced to 10 components covering 95.27% of the variance. PCA improved only SVM, left Logistic Regression and Stacking unchanged, and reduced accuracy for the remaining seven models, most noticeably AdaBoost. The stacked ensemble was the most resistant to the change, as its accuracy did not change at all despite its individual base learners responding differently. Overall, PCA is not a universally helpful preprocessing step here, its benefit depends heavily on the type of model being used.

13 References

1. I. T. Jolliffe, “Principal Component Analysis,” Springer Series in Statistics, 2002.
2. L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
3. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proc. 22nd ACM SIGKDD, pp. 785–794, 2016.
4. Scikit-learn Developers, “Decomposition: PCA,” [Online]. Available: <https://scikit-learn.org/stable/modules/decomposition.html#pca>